

[HP06] "Computer Architecture: A Quantitative Approach"

John Hennessy and David Patterson

Morgan-Kaufmann, 2006

A great and marvelous book about computer architecture. Read it!

[H87] "Aspects of Cache Memory and Instruction Buffer Performance"

Mark D. Hill

Ph.D. Dissertation, U.C. Berkeley, 1987

Mark Hill, in his dissertation work, introduced the Three C's, which later gained wide popularity with its inclusion in H&P [HP06]. The quote from therein: "I have found it useful to partition misses ... into three components intuitively based on the cause of the misses (page 49)."

[KE+62] "One-level Storage System"

T. Kilburn, and D.B.G. Edwards and M.J. Lanigan and F.H. Sumner

IRE Trans. EC-11:2, 1962

Although Atlas had a use bit, it only had a very small number of pages, and thus the scanning of the use bits in large memories was not a problem the authors solved.

[M+70] "Evaluation Techniques for Storage Hierarchies"

R. L. Mattson, J. Gecsei, D. R. Slutz, I. L. Traiger

IBM Systems Journal, Volume 9:2, 1970

A paper that is mostly about how to simulate cache hierarchies efficiently; certainly a classic in that regard, as well for its excellent discussion of some of the properties of various replacement algorithms. Can you figure out why the stack property might be useful for simulating a lot of different-sized caches at once?

[MM03] "ARC: A Self-Tuning, Low Overhead Replacement Cache"

Nimrod Megiddo and Dharmendra S. Modha

FAST 2003, February 2003, San Jose, California

An excellent modern paper about replacement algorithms, which includes a new policy, ARC, that is now used in some systems. Recognized in 2014 as a "Test of Time" award winner by the storage systems community at the FAST '14 conference.

Homework

This simulator, `paging-policy.py`, allows you to play around with different page-replacement policies. See the README for details.

Questions

1. Generate random addresses with the following arguments: `-s 0 -n 10`, `-s 1 -n 10`, and `-s 2 -n 10`. Change the policy from FIFO, to LRU, to OPT. Compute whether each access in said address traces are hits or misses.
2. For a cache of size 5, generate worst-case address reference streams for each of the following policies: FIFO, LRU, and MRU (worst-case reference streams cause the most misses possible. For the worst case reference streams, how much bigger of a cache is needed to improve performance dramatically and approach OPT?
3. Generate a random trace (use python or perl). How would you expect the different policies to perform on such a trace?
4. Now generate a trace with some locality. How can you generate such a trace? How does LRU perform on it? How much better than RAND is LRU? How does CLOCK do? How about CLOCK with different numbers of clock bits?
5. Use a program like `valgrind` to instrument a real application and generate a virtual page reference stream. For example, running `valgrind --tool=lackey --trace-mem=yes ls` will output a nearly-complete reference trace of every instruction and data reference made by the program `ls`. To make this useful for the simulator above, you'll have to first transform each virtual memory reference into a virtual page-number reference (done by masking off the offset and shifting the resulting bits downward). How big of a cache is needed for your application trace in order to satisfy a large fraction of requests? Plot a graph of its working set as the size of the cache increases.